

Capítulo 2 — La VPS: aprovisionar, entrar y cerrar

Unidad de estudio · Edición ampliada con fundamentos teóricos

2.1. Alcance de la clase	2
2.2. Por qué existe un servidor virtual: origen y diseño de la virtualización	2
2.3. Qué es un VPS y dónde se ubica entre las alternativas	4
2.3.1. Requisitos del práctico	4
2.4. Aprovisionamiento.....	5
2.4.1. Verificación cruzada con la Clase 1	6
2.5. Acceso remoto: el protocolo SSH	6
2.5.1. Anatomía de una conexión SSH	6
2.5.2. Generación del par de claves.....	7
2.5.3. Instalación de las claves del grupo.....	7
2.5.4. Conexión.....	8
2.5.5. Deshabilitación del acceso por contraseña	8
2.6. Puertos y sockets: qué significa "escuchar"	8
2.6.1. El socket como tupla	8
2.6.2. Rangos de puertos y su asignación	9
2.6.3. Inspección del estado actual	9
2.7. El firewall: anatomía de netfilter	10
2.7.1. El recorrido de un paquete	11
2.7.2. Configuración con ufw	12
2.8. El problema: Docker no respeta el firewall	12
2.8.1. Por qué ocurre, en términos de netfilter	13
2.8.2. Verificación desde el exterior	14
2.8.3. Cómo se evita el problema	14
2.9. Riesgos concretos de la exposición innecesaria	15
2.9.1. El descubrimiento es automático e inmediato	15
2.9.2. Qué se arriesga con cada puerto	15
2.9.3. Consecuencias que exceden al servidor	16
2.10. Publicación segura del panel de administración	16
2.11. Endurecimiento mínimo.....	17
2.11.1. fail2ban.....	18
2.12. Principios de seguridad y evolución de las herramientas	18
2.12.1. Tres principios que explican todo el capítulo	18
2.12.2. De iptables a nftables.....	19
2.12.3. Por qué Ed25519 y no RSA	19

2.13. Verificación	20
2.14. Errores frecuentes	20
2.15. Actividades	21
2.16. Síntesis	21
2.17. Referencias y lecturas complementarias	22

2.1. Alcance de la clase

Este capítulo cubre la puesta en marcha del servidor donde vivirá la aplicación. El énfasis no está en el aprovisionamiento —que en la práctica se reduce a unos pocos clics— sino en lo que ocurre inmediatamente después: **un servidor recién creado es, por defecto, un objetivo**.

Como en el capítulo anterior, los pasos operativos son pocos y su ejecución es breve. Lo que exige tiempo es entender qué se está haciendo. Detrás de "abrir un puerto" hay un modelo de comunicación entre procesos que se remonta a los años setenta; detrás de "activar el firewall" hay un subsistema del núcleo de Linux con un recorrido de paquetes que conviene conocer, porque es exactamente ese recorrido el que explica el problema más grave y menos difundido de todo el módulo: **Docker publica puertos que el firewall no ve**. No se puede diagnosticar ese problema sin el modelo, y no se puede confiar en un servidor sin poder diagnosticarlo.

Al finalizar la clase, cada grupo debe contar con un VPS operativo, accesible por claves SSH, con el firewall configurado y el panel de administración publicado bajo su propio dominio con certificado válido.

Contenidos

1. Origen y objetivos de diseño de la virtualización de servidores.
2. Qué es un VPS y qué lo distingue de otras modalidades de alojamiento.
3. Aprovisionamiento en Hostinger con plantilla Easypanel.
4. El protocolo SSH: arquitectura, autenticación y verificación del anfitrión.
5. Puertos y sockets: qué significa que un proceso "escuche".
6. Anatomía de netfilter y el recorrido de un paquete.
7. Configuración del firewall con `ufw`.
8. El problema de Docker y el firewall, explicado desde el recorrido del paquete.
9. Riesgos concretos de la exposición innecesaria de puertos.
10. Publicación segura del panel de administración.
11. Endurecimiento mínimo del servidor.
12. Principios de diseño en seguridad y evolución de las herramientas.

2.2. Por qué existe un servidor virtual: origen y diseño de la virtualización

Un servidor privado virtual no es un tipo de computadora: es una **ilusión cuidadosamente construida**. El grupo recibe lo que parece una máquina completa —con su propio sistema operativo, su propia dirección IP, su propio disco y su propio administrador— cuando en realidad está compartiendo hierro con decenas de clientes desconocidos. Entender cómo se produce esa ilusión, y dónde tiene fisuras, explica varias cosas que después se observan en la práctica: por qué el servidor a veces va más lento sin motivo aparente, por qué reinstalar el sistema operativo tarda tres minutos

en lugar de media hora, y por qué el proveedor puede vender por diez dólares algo que hace veinte años costaba miles.

El problema es viejo y el punto de partida fue económico. En los años sesenta, una computadora central costaba una fortuna y se usaba de a un programa por vez; el tiempo de máquina ocioso era dinero quemado. IBM atacó el problema en su centro de investigación de Cambridge con los sistemas CP-40 y CP-67, y luego con VM/370 en 1972: en lugar de repartir el tiempo de una única máquina entre varios usuarios —lo que ya hacían los sistemas de tiempo compartido—, el programa de control creaba **varias máquinas virtuales completas**, cada una con la ilusión de tener el hardware para sí sola. La diferencia es sutil y decisiva: en el tiempo compartido los usuarios conviven dentro de un mismo sistema operativo y se ven entre sí; en la virtualización cada uno tiene su propio sistema operativo y no sabe que los demás existen.

En 1974, Gerald Popek y Robert Goldberg formalizaron el problema en un artículo que sigue siendo la referencia: enunciaron las tres propiedades que debe cumplir un sistema para llamarse virtualizador —**equivalencia** (los programas se comportan igual que sobre el hardware real), **control de recursos** (el hipervisor tiene la última palabra sobre el hardware) y **eficiencia** (la mayoría de las instrucciones se ejecutan directamente sobre el procesador, sin emulación)— y demostraron formalmente qué condición debe cumplir un juego de instrucciones para admitirlas. La arquitectura x86, la de las computadoras personales, **no cumplía esa condición**. Ese incumplimiento es la razón por la que la virtualización tardó treinta años en llegar al mundo de los servidores comunes: primero mediante trucos de traducción binaria en tiempo de ejecución (VMware, 1999), luego modificando el sistema operativo huésped para que colaborase (Xen, 2003), y finalmente con soporte del propio procesador —las extensiones VT-x de Intel y AMD-V, alrededor de 2006— que es lo que se usa hoy y lo que hace que un VPS rinda casi como una máquina real.

Sobre esa base, la industria distingue dos ubicaciones posibles para el **hipervisor**, el programa que crea y arbitra las máquinas virtuales.

Tipo	Dónde corre	Ejemplos	Uso típico
Tipo 1 (nativo)	Directamente sobre el hardware	KVM, Xen, VMware ESXi, Hyper-V	Proveedores de nube y VPS
Tipo 2 (alojado)	Como un programa más dentro de un sistema operativo	VirtualBox, VMware Workstation	Escritorio, laboratorio

El VPS del práctico corre sobre un hipervisor de tipo 1, casi con seguridad **KVM**, que desde 2007 forma parte del propio núcleo de Linux: la máquina física ejecuta Linux, y Linux mismo actúa como hipervisor. De esa arquitectura se desprenden tres consecuencias que conviene tener presentes.

El aislamiento es real, pero los recursos se comparten. El sistema operativo del grupo no puede ver ni tocar el de otro cliente: son espacios de memoria separados que el procesador hace cumplir por hardware. Pero el procesador físico, el disco y la placa de red sí son los mismos. Los proveedores practican habitualmente el **sobreaprovisionamiento**: venden más núcleos virtuales que núcleos reales, apostando a que no todos los clientes estarán al máximo a la vez. Cuando esa apuesta falla, aparece el fenómeno del *vecino ruidoso*: el servidor se pone lento sin que nada haya cambiado del lado del grupo.

El estado completo de la máquina es un archivo. Como el disco virtual es una imagen y la configuración es un registro en una base de datos del proveedor, destruir y recrear un servidor es una operación de archivos, no de hardware. Por eso "cambiar el sistema operativo" del VPS tarda minutos, y por eso la operación **es destructiva y no tiene deshacer**: no se está actualizando nada, se está tirando la imagen anterior y escribiendo otra encima.

La responsabilidad se corta en un punto preciso. El proveedor responde por el hipervisor, el hardware y la red. Todo lo que ocurre dentro del sistema operativo huésped —usuarios, puertos, actualizaciones, servicios expuestos— es del titular. Ese corte no es una postura comercial: es la consecuencia directa de que el proveedor, por diseño, no puede ver adentro.

PARA ENTENDER

Quedate con esto: **el VPS es barato porque estás compartiendo hierro, y es tuyo porque el aislamiento lo hace cumplir el procesador.** Las dos cosas son ciertas al mismo tiempo, y de ahí sale todo lo demás. Si se pone lento sin motivo, mirá al vecino. Si queda comprometido, mirate al espejo: adentro de tu sistema operativo el proveedor no tiene ni jurisdicción ni visibilidad.

2.3. Qué es un VPS y dónde se ubica entre las alternativas

Un **servidor privado virtual** es una porción aislada de un servidor físico, con su propio sistema operativo, sus propios recursos asignados y acceso administrativo completo. La tabla siguiente lo sitúa respecto de las alternativas, ordenadas por cuánto se administra y cuánto se delega.

Modalidad	Qué se administra	Qué se delega	Costo
Alojamiento compartido	Archivos y base de datos	Todo lo demás	Muy bajo
PaaS (Vercel, Railway)	La aplicación	Sistema operativo, red, escalado	Bajo a medio
VPS	Todo el sistema operativo	Solo el hardware	Medio
Servidor dedicado	Todo, incluido el hardware	Nada	Alto

Obsérvese que la tabla describe un único eje: **cuánta abstracción hay entre quien desarrolla y la máquina.** Cada escalón hacia arriba elimina trabajo operativo y, con él, elimina también visibilidad y control. Un alojamiento compartido no permite elegir versión de Python; un PaaS no permite ver la tabla de reglas del firewall porque no hay tal tabla expuesta al cliente.

La elección del VPS para este módulo es deliberada y va exactamente contra la comodidad. Un PaaS resolvería el despliegue en menos pasos, pero ocultaría precisamente aquello que se pretende enseñar: la red, los puertos, el proxy inverso, los certificados y el aislamiento entre servicios. En un PaaS esas piezas existen —alguien las configuró— pero no son observables, y lo que no se observa no se aprende. El VPS es el nivel de abstracción más bajo en el que todavía se puede trabajar en una cursada, y el más alto en el que todas las piezas siguen siendo visibles.

PARA ENTENDER

Con un VPS pasás a ser el administrador del sistema. Eso significa que si el servidor queda comprometido, **es tu responsabilidad**, no la del proveedor. Hostinger te vende una máquina; lo que hagas con ella es tu problema. Esa es exactamente la razón por la que la mitad de esta clase es sobre seguridad y no sobre despliegue.

2.3.1. Requisitos del práctico

Recurso	Mínimo	Motivo
Memoria RAM	2 GB	Requisito declarado por Easypanel

Almacenamiento	20 GB	Imágenes de Docker y sus capas
Sistema operativo	Ubuntu 24.04	Base de la plantilla de Easypanel
Puertos 80 y 443	Libres	Traefik toma control exclusivo del enrutamiento HTTP

2.4. Aprovisionamiento

Hostinger ofrece una plantilla de sistema operativo con Easypanel preinstalado sobre Ubuntu 24.04, que evita la instalación manual. Lo que la plantilla hace, en términos de la sección anterior, es escribir una imagen de disco preparada de antemano en lugar de la imagen vacía que traía el servidor: por eso el proceso es tan rápido y por eso es irreversible.

Procedimiento:

1. Acceder a hPanel y seleccionar el servidor en la sección **VPS**.
2. Ingresar a la opción de cambio de sistema operativo (*Operating System*).
3. Seleccionar la plantilla **Easypanel** y confirmar.
4. Aguardar la finalización del proceso, que demora algunos minutos.

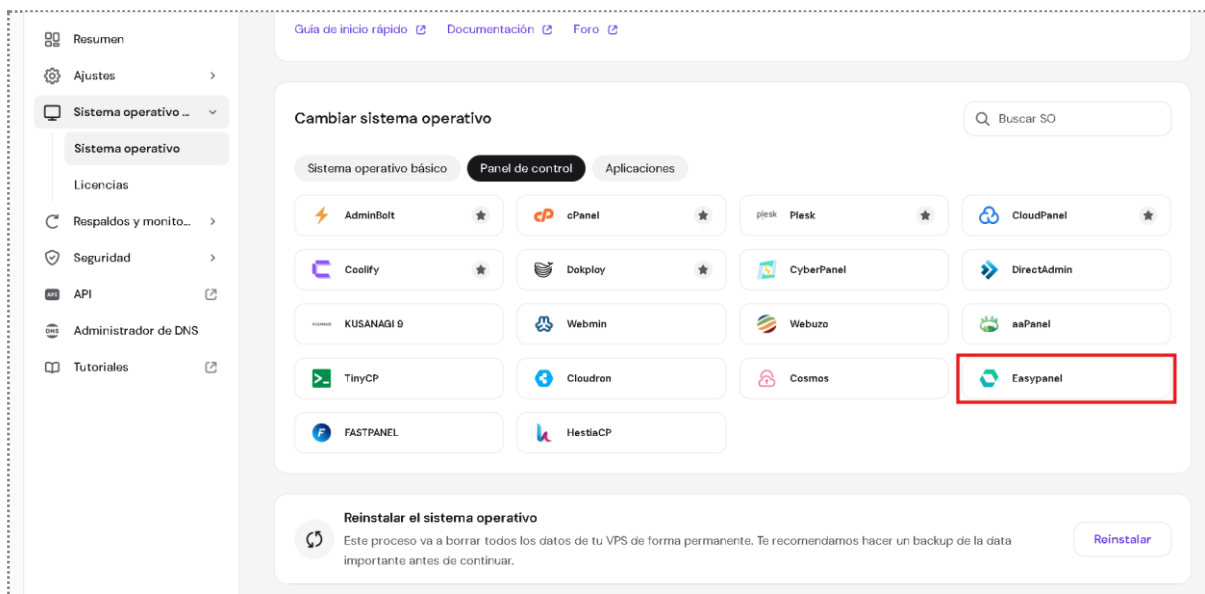


Figura 2.1. Selección de la plantilla con Easypanel preinstalado en hPanel.

⚠ OJO ACÁ

El cambio de sistema operativo **borra todo el contenido del servidor de forma irreversible**. En un VPS recién contratado no hay problema. Si el grupo ya venía usándolo para otra cosa, hagan copia de seguridad primero. No hay deshacer.

Al finalizar, registrar dos datos que se utilizan de inmediato:

- **La dirección IP pública del servidor.** Es la que se cargó en los registros DNS de la Clase 1. Verificar que coincida.
- **La contraseña de root,** si el proveedor la generó. Se utiliza una sola vez, para configurar el acceso por clave.

2.4.1. Verificación cruzada con la Clase 1

Antes de continuar, comprobar que el dominio configurado la clase anterior apunta efectivamente a este servidor:

```
nslookup loquesea.tudominio.com
```

La dirección devuelta debe ser idéntica a la IP del VPS recién aprovisionado. Si no lo es, corregir el registro comodín en el panel DNS antes de avanzar. Nótese que el nombre `loquesea` no existe en ninguna parte: se está ejerciendo el comodín de la sección 1.12.1, y que la consulta funcione es en sí mismo una verificación del capítulo anterior.

2.5. Acceso remoto: el protocolo SSH

SSH (*Secure Shell*) es el protocolo mediante el cual se obtiene una consola remota del servidor. Su origen es una respuesta directa a un problema concreto: hasta mediados de los noventa, el acceso remoto se hacía con Telnet y `rlogin`, protocolos que transmitían **la contraseña en texto plano** por la red. En 1995, tras un episodio de captura de contraseñas en la red de su universidad, Tatu Ylönen escribió la primera versión de SSH. La versión 2 del protocolo, incompatible con la primera y la única en uso hoy, quedó normada en 2006 en las RFC 4251 a 4254.

2.5.1. Anatomía de una conexión SSH

La arquitectura de SSH separa tres capas que se establecen en orden, y distinguirlas explica dos comportamientos que suelen confundirse.

Capa de transporte (RFC 4253). Establece un canal cifrado e íntegro, y —esto es lo importante— **autentica al servidor ante el cliente**, no al revés. El servidor presenta su *clave de anfitrión* (*host key*) y demuestra que la posee. Recién sobre ese canal ya seguro ocurre todo lo demás.

Capa de autenticación (RFC 4252). Ahora sí, el cliente demuestra quién es. Es acá donde se elige entre contraseña y clave pública.

Capa de conexión (RFC 4254). Multiplexa sobre el mismo canal la sesión interactiva, la copia de archivos y los túneles de puertos.

La autenticación del servidor merece un párrafo propio porque produce un mensaje que todo el mundo ve y casi nadie lee. La primera vez que se conecta a un servidor, el cliente no tiene forma de saber si esa clave de anfitrión es la legítima: pregunta, y si el usuario acepta, la guarda en `~/.ssh/known_hosts`. Este modelo se llama **confianza en el primer uso** (*trust on first use*, TOFU) y es un compromiso deliberado: es vulnerable exactamente una vez —la primera— y a partir de ahí detecta cualquier suplantación. Por eso, si alguna vez aparece la advertencia grande de que la clave del anfitrión cambió, **no es un trámite**: o el servidor se reinstaló, o alguien se está interponiendo en el camino.

La autenticación del cliente admite dos formas, y la diferencia entre ellas no es de comodidad sino de seguridad.

	Contraseña	Clave pública
Qué viaja por la red	Un secreto reutilizable	Nada secreto
Resistencia a fuerza bruta	Baja	Prácticamente total
Qué pasa si se filtra el servidor	El atacante obtiene acceso	Nada: la clave privada nunca estuvo ahí
Cómo se revoca el acceso a una persona	Cambiando la contraseña de todos	Borrando una línea

El mecanismo de la clave pública conviene enunciarlo con precisión, porque la frase "se copia la clave al servidor" induce a error. La autenticación por clave es un **desafío criptográfico**: el servidor envía al cliente un dato aleatorio, el cliente lo firma con su clave privada, y el servidor verifica esa firma con la clave pública que tiene guardada. La clave privada **nunca se transmite**, ni siquiera cifrada. De ahí la propiedad más importante de la fila tercera de la tabla: si el servidor entero cae en manos ajenas, el atacante obtiene el archivo `authorized_keys` con las claves públicas, que no le sirven para absolutamente nada.

2.5.2. Generación del par de claves

En el equipo del alumno, no en el servidor:

```
ssh-keygen -t ed25519 -C "apellido-prog3"
```

El comando genera dos archivos en `C:\Users\usuario\.ssh\` (Windows) o `~/ .ssh/` (Linux y macOS):

Archivo	Qué es	Se comparte
<code>id_ed25519</code>	Clave privada	Nunca
<code>id_ed25519.pub</code>	Clave pública	Sí, se copia al servidor

⚠ OJO ACÁ

La que termina en `.pub` es la que se comparte. La otra **no sale nunca de tu máquina**: ni por WhatsApp, ni por el campus, ni en un repositorio. Si alguna vez subís una clave privada a un repositorio de GitHub, dala por comprometida y generá un par nuevo. Hay bots que escanean GitHub buscando exactamente eso, y tardan minutos.

2.5.3. Instalación de las claves del grupo

Como cada VPS es compartida por 3 o 4 integrantes, **cada uno instala su propia clave pública**. Ese es justamente el beneficio del esquema: cuatro accesos independientes, revocables por separado, sin ninguna contraseña compartida. Conviene notar que esto no es solo comodidad: una contraseña compartida por cuatro personas es, a efectos de auditoría, una identidad sin dueño. Con claves, cada línea del archivo tiene nombre y apellido.

En hPanel, sección **SSH Keys** del VPS, se agrega el contenido de cada archivo `.pub`.

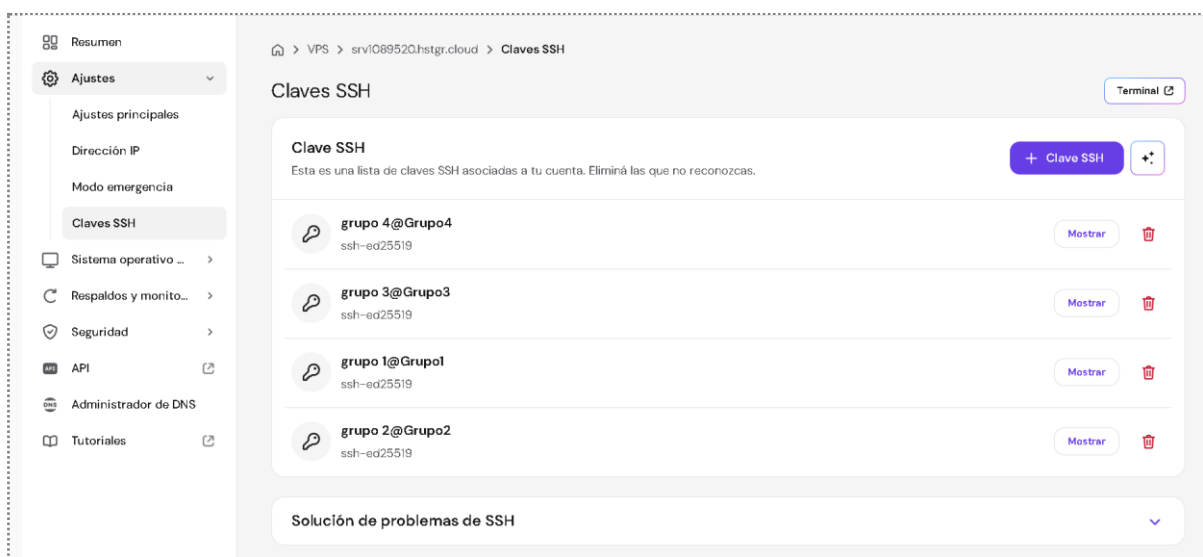


Figura 2.2. Cuatro claves públicas independientes: un acceso por integrante, revocable por separado.

Alternativamente, desde una sesión ya abierta en el servidor, se agrega cada clave como una línea del archivo `~/.ssh/authorized_keys`.

2.5.4. Conexión

```
ssh root@tudominio.com
```

Nótese que se utiliza el dominio, no la dirección IP: los registros de la Clase 1 ya lo permiten. La primera conexión solicita confirmar la huella del servidor —el modelo TOFU de la sección 2.5.1— que queda registrada localmente para detectar suplantaciones futuras.

2.5.5. Deshabilitación del acceso por contraseña

Una vez verificado que las cuatro claves funcionan, se desactiva la autenticación por contraseña editando `/etc/ssh/sshd_config`:

```
PasswordAuthentication no
PermitRootLogin prohibit-password
```

Y se recarga el servicio:

```
sudo systemctl reload ssh
```

La segunda directiva merece una lectura atenta: `prohibit-password` no prohíbe entrar como root, prohíbe entrar como root **con contraseña**. El acceso por clave sigue permitido. Es el valor predeterminado en las distribuciones modernas y es el adecuado para este práctico, donde el trabajo administrativo es constante.

⚠ OJO ACÁ

Verificá que las claves funcionen ANTES de deshabilitar la contraseña. Abrí una segunda terminal, conectate con la clave, y recién ahí tocá el archivo. Si te equivocás y cerrás la única sesión que tenías, quedás afuera de tu propio servidor y hay que entrar por la consola de emergencia de Hostinger. Le pasa a todo el mundo una vez. Que no sea hoy.

2.6. Puertos y sockets: qué significa "escuchar"

2.6.1. El socket como tupla

Una dirección IP identifica una máquina. Un **puerto** identifica un servicio dentro de esa máquina. Es un número de 16 bits, con lo cual el rango va de 1 a 65535.

Pero la formulación precisa es más útil que esa aproximación, y explica varias cosas que de otro modo parecen mágicas. En la pila TCP/IP, una conexión establecida no se identifica por un puerto sino por una **tupla de cuatro elementos**: dirección de origen, puerto de origen, dirección de destino y puerto de destino. Dos conexiones distintas al mismo servidor y al mismo puerto **no se confunden** porque difieren en el par de origen. Por eso un servidor web con un único puerto 443 puede atender diez mil conexiones simultáneas: no hay diez mil puertos, hay diez mil tuplas distintas.

Cuando un proceso se pone a **escuchar** en un puerto, le está diciendo al sistema operativo: *entregame a mí toda conexión nueva dirigida a este número*. Ese socket en escucha es un objeto de un tipo distinto al de una conexión establecida —está a medio especificar, con el origen todavía vacío— y cada conexión aceptada genera un socket nuevo, ya completo. Dos procesos no pueden escuchar simultáneamente en la misma combinación de puerto e interfaz.

Además del puerto, un proceso declara **en qué interfaz** escucha. Esta distinción es la misma que aparece en el `Dockerfile` del backend y se retoma en la Clase 3:

Interfaz	Significado	Quién puede conectarse
127.0.0.1	Solo la máquina local	Únicamente procesos del mismo equipo
0.0.0.0	Todas las interfaces	Cualquiera que llegue por la red



PARA ENTENDER

Esta tabla parece un detalle de configuración y es una decisión de seguridad de primer orden. Un servicio que escucha en 127.0.0.1 **es inalcanzable desde internet aunque el firewall esté apagado y el puerto abierto**: no hay firewall que valga, simplemente no hay nadie escuchando del lado de afuera. Es la forma más barata y más robusta de cerrar algo, y la vas a volver a usar en la Clase 3 y en la Clase 5.

2.6.2. Rangos de puertos y su asignación

Los 65535 puertos no son todos iguales. La IANA, el organismo que administra los recursos de numeración de internet, los divide en tres rangos con procedimientos distintos, normados en la RFC 6335.

Rango	Nombre	Quién los asigna	Ejemplos
0 – 1023	Bien conocidos	IANA, por trámite formal	22 SSH · 53 DNS · 80 HTTP · 443 HTTPS
1024 – 49151	Registrados	IANA, por trámite simplificado	3000, 5432 PostgreSQL, 6379 Redis
49152 – 65535	Dinámicos o efímeros	Nadie: los toma el sistema operativo	El puerto de origen de tu navegador

Dos consecuencias prácticas. La primera: en los sistemas Unix, **escuchar en un puerto por debajo de 1024 requiere privilegios de administrador**. Es una restricción histórica pensada para que un usuario cualquiera no pudiera hacerse pasar por el servidor web de la máquina, y es la razón por la que los servidores de aplicaciones escuchan en 8000, 3000 o 5000 y hay un proxy inverso adelante ocupando el 80 y el 443. La segunda: la asignación es **una convención, no una imposición técnica**. Nada impide correr un servidor SSH en el puerto 2222; lo que se pierde es que las herramientas lo encuentren por omisión.

2.6.3. Inspección del estado actual

```
sudo ss -tlnp
```

Bandera	Significado
-t	Solo TCP
-l	Solo sockets en escucha
-n	Números, sin resolver nombres
-p	Mostrar el proceso responsable

La bandera **-l** es la que conecta con la sección 2.6.1: pide específicamente los sockets en estado LISTEN, es decir, los que están a la espera de conexiones nuevas. Sin esa bandera, **ss** muestra además todas las conexiones ESTABLISHED en curso, que son muchas más y responden otra pregunta.

```

https://ubuntu.com/engage/secure-kubernetes-at-the-edge
Expanded Security Maintenance for Applications is not enabled.
64 updates can be applied immediately.
To see these additional updates run: apt list --upgradable
14 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

1 updates could not be installed automatically. For more details,
see /var/log/unattended-upgrades/unattended-upgrades.log

*** System restart required ***
Last login: Thu May 28 13:27:32 2026 from [REDACTED]
root@srv1089520:~# sudo ss -tlnp
State Recv-Q Send-Q Local Address:Port Peer Address:Port Process
LISTEN 0 4096 0.0.0.0:27017 0.0.0.0:* users:(("docker-proxy",pid=4738,fd=7))
LISTEN 0 511 127.0.0.1:18791 0.0.0.0:* users:(("MainThread",pid=2310578,fd=27))
LISTEN 0 511 127.0.0.1:18789 0.0.0.0:* users:(("MainThread",pid=2310578,fd=25))
LISTEN 0 4096 127.0.0.54:53 0.0.0.0:* users:(("systemd-resolve",pid=2180501,fd=17))
LISTEN 0 4096 0.0.0.0:3000 0.0.0.0:* users:(("docker-proxy",pid=4967,fd=7))
LISTEN 0 4096 0.0.0.0:443 0.0.0.0:* users:(("docker-proxy",pid=4891,fd=7))
LISTEN 0 4096 0.0.0.0:22 0.0.0.0:* users:(("sshd",pid=2023303,fd=3),("systemd",pid=1,fd=182))
LISTEN 0 4096 0.0.0.0:80 0.0.0.0:* users:(("docker-proxy",pid=4873,fd=7))

```

Figura 2.3. Puertos en escucha y proceso responsable de cada uno.

En un servidor recién aprovisionado con la plantilla, la salida típica incluye:

Puerto	Proceso	Función
22	sshd	Acceso administrativo
80	Traefik	HTTP, y validación de certificados
443	Traefik	HTTPS
3000	Easypanel	Panel de administración

EXPERIMENTO

Corré `ss -tlnp` y leé la salida línea por línea. Cada puerto abierto es **un proceso concreto que alguien decidió levantar**, no una propiedad mágica del servidor.

Y ahora respondete: de estos cuatro, ¿cuáles necesita ver el mundo entero? La respuesta —80 y 443— es toda la clase de seguridad resumida.

Corré después el mismo comando sin la `-l` y mirá cuánto crece la salida. Esa diferencia son las conexiones ya establecidas: el socket en escucha es uno, las conversaciones en curso son muchas.

2.7. El firewall: anatomía de netfilter

Un firewall decide qué paquetes entran y cuáles se descartan, antes de que lleguen al proceso que escucha. En Linux esa función no la cumple un programa sino un subsistema del propio núcleo, llamado **netfilter**, y las herramientas de línea de comandos —`iptables`, `nft`, `ufw`— no son el firewall: son formas de escribirle reglas. Confundir la herramienta con el subsistema es exactamente el error que produce el problema de la sección 2.8, así que vale la pena dedicarle esta sección.

2.7.1. El recorrido de un paquete

Netfilter define **puntos de enganche** (*hooks*) en lugares precisos del recorrido que un paquete hace dentro del núcleo. En cada punto se evalúan las reglas que allí haya registradas. Los cinco puntos son:

Punto de enganche	Cuándo se evalúa
PREROUTING	Apenas llega el paquete, antes de decidir a dónde va
INPUT	El paquete va a un proceso de esta misma máquina
FORWARD	El paquete atraviesa la máquina hacia otro destino
OUTPUT	El paquete lo generó un proceso local
POSTROUTING	Justo antes de salir por la placa de red

La pieza decisiva está entre PREROUTING y los dos siguientes: **la decisión de enrutamiento**. Después de PREROUTING, el núcleo mira la dirección de destino del paquete y se pregunta si es para él o para otro. Si es para él, el paquete sigue por INPUT. Si es para otro, sigue por FORWARD. **Son dos caminos excluyentes**: un paquete pasa por uno o por el otro, nunca por los dos.

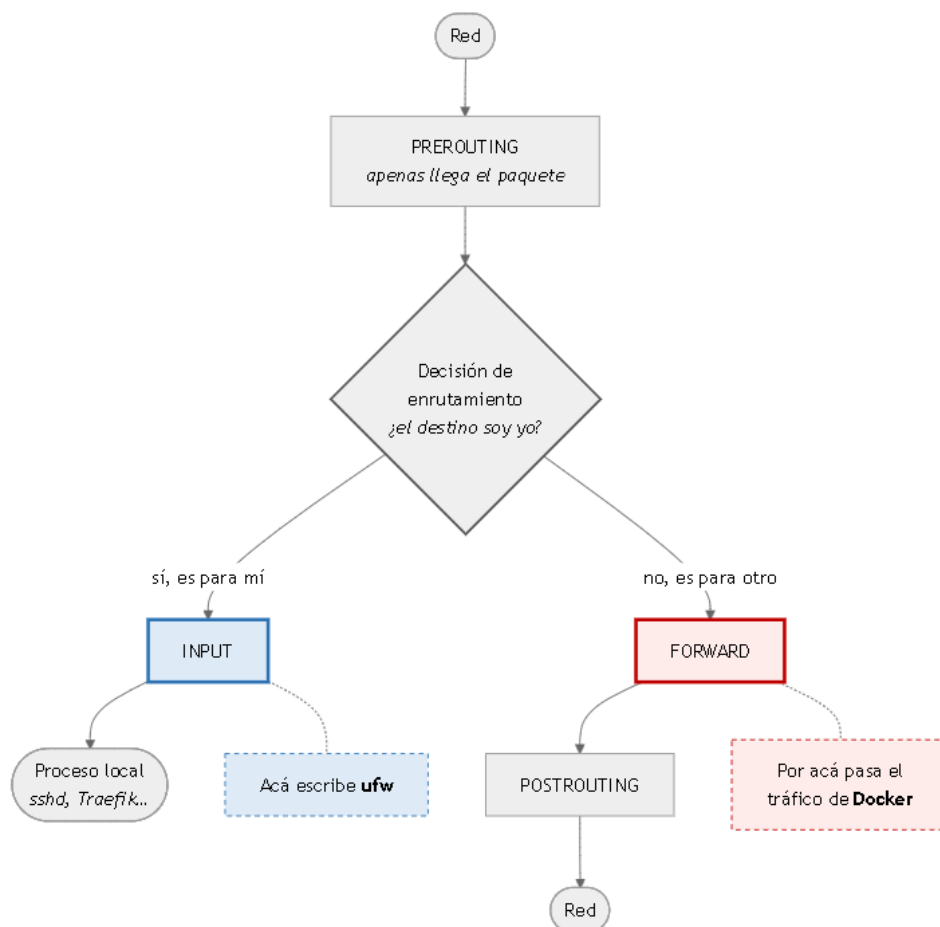


Figura 2.4. Recorrido de un paquete por los puntos de enganche de netfilter. La decisión de enrutamiento separa dos caminos excluyentes: ufw escribe en uno y Docker pasa por el otro.

Reténgase este esquema, porque toda la sección 2.8 es una consecuencia suya: **ufw escribe sus reglas en INPUT**; el tráfico hacia los contenedores de Docker pasa por FORWARD.

Netfilter organiza además las reglas en **tablas** según su propósito: filter para aceptar o descartar, nat para reescribir direcciones, mangle para modificar campos. La traducción de direcciones que hace Docker vive en nat, y es lo que convierte "el puerto 5432 de la máquina" en "el puerto 5432 del contenedor tal", que tiene otra dirección IP.

2.7.2. Configuración con ufw

ufw (*Uncomplicated Firewall*) es una capa de conveniencia sobre lo anterior: traduce órdenes legibles a reglas de netfilter. La configuración correcta parte de una **política restrictiva** —descartar todo lo que no esté explícitamente permitido— y abre solo lo necesario. Ese orden no es casual: es la aplicación directa del principio de *valores predeterminados seguros* que se enuncia en la sección 2.12.1.

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
```

Verificación:

```
sudo ufw status verbose
```

Puerto	Estado	Justificación
22	Abierto	Sin esto se pierde el acceso administrativo
80	Abierto	Redirección a HTTPS y validación de Let's Encrypt
443	Abierto	Todo el tráfico de la aplicación
Todo lo demás	Cerrado	No hay ningún motivo para exponerlo

OJO ACÁ

El orden importa. Si corrés `ufw enable` **antes** de permitir el 22, el firewall corta tu propia sesión SSH en el acto y quedás afuera. Permitir primero, habilitar después. Siempre.

DATO

El puerto 80 no se puede cerrar aunque la aplicación funcione toda por HTTPS. Let's Encrypt valida la titularidad del dominio pidiendo un archivo por el puerto 80. Sin ese puerto abierto no hay certificado, y sin certificado no hay HTTPS.

2.8. El problema: Docker no respeta el firewall

Esta sección es, probablemente, la más importante del capítulo, y es la que menos aparece en los tutoriales de despliegue. Ahora que está sobre la mesa el recorrido del paquete de la sección 2.7.1, el problema deja de ser un misterio y pasa a ser una consecuencia previsible.

2.8.1. Por qué ocurre, en términos de netfilter

Un contenedor **no comparte la pila de red del anfitrión**: tiene su propia interfaz y su propia dirección IP privada, colgada de un puente virtual que Docker crea al instalarse. Desde el punto de vista del núcleo, un contenedor es *otra máquina* alcanzable a través del anfitrión.

Seguí ahora el recorrido de un paquete que llega desde internet al puerto publicado de un contenedor:

1. Entra por PREROUTING. Ahí Docker tiene una regla en la tabla `nat` que **reescribe la dirección de destino**: donde decía "la IP pública del VPS, puerto 5432", ahora dice "la IP privada del contenedor, puerto 5432".
2. Llega la decisión de enrutamiento. El núcleo mira el destino —que ya fue reescrito— y concluye que **no es para él**: es para el contenedor.
3. En consecuencia, el paquete sigue por FORWARD, donde Docker registró sus cadenas DOCKER y DOCKER-USER.
4. **Nunca pasa por INPUT**. Y las reglas de `ufw` viven en INPUT.

La consecuencia es contraintuitiva y grave:

```
sudo ufw deny 5432
```

Un contenedor levantado con `-p 5432:5432` queda **accesible desde internet igualmente**, y `sudo ufw status` informa que el puerto está bloqueado. Las dos cosas son ciertas: `ufw` efectivamente escribió esa regla, y esa regla efectivamente no se evalúa nunca para ese tráfico.

Lo que se cree	Lo que ocurre
<code>ufw</code> protege todos los puertos	<code>ufw</code> no ve el tráfico dirigido a puertos publicados por Docker
<code>ufw status</code> refleja el estado real	Refleja únicamente las reglas de <code>ufw</code>
Un puerto no listado está cerrado	Puede estar abierto por Docker

Conviene subrayar que **esto no es un error de Docker ni una mala configuración de Ubuntu**. Es la interacción previsible de dos programas que escriben reglas en el mismo subsistema sin conocerse entre sí, en puntos de enganche distintos. Docker documenta la cadena DOCKER-USER precisamente como el lugar previsto para que el administrador intervenga; lo que no hace es avisar que `ufw` no lo usa.

⚠ OJO ACÁ

Leé esto dos veces: **`ufw status` te puede mentir**. No es un bug ni una mala configuración, es cómo funciona Docker. Un montón de bases de datos expuestas a internet en todo el mundo están así porque alguien corrió `ufw deny` y se quedó tranquilo.

La verificación de verdad no es `ufw status`. Es escanear el servidor **desde afuera**.

💡 PARA ENTENDER

Fijate en la forma del error, porque se repite en toda la ingeniería: **una herramienta te informó sobre sí misma y vos lo leíste como información sobre el sistema**. `ufw status` no dice "el puerto 5432 está cerrado"; dice "yo tengo una regla que lo cierra". Son dos afirmaciones distintas y solo

la segunda es la que ufw puede sostener. Cada vez que una herramienta te reporte un estado, preguntate si te está hablando del mundo o de su propia configuración.

2.8.2. Verificación desde el exterior

La única comprobación confiable consiste en consultar los puertos desde otra máquina. La razón es epistemológica antes que técnica: cualquier herramienta que corra **dentro** del servidor está describiendo lo que ella cree; solo una prueba desde afuera atraviesa el mismo camino que atravesaría un atacante.

```
nmap -Pn tudominio.com
```

Si nmap no está disponible, servicios como `nmap.online` o `canyouseeme.org` cumplen la misma función desde el navegador.

```
root@srv1089520:~# nmap -Pn 72.61.33.27
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-07-29 14:22 UTC
Nmap scan report for srv1089520.hstgr.cloud (72.61.33.27)
Host is up (0.000010s latency).
Not shown: 995 closed tcp ports (reset)
PORT      STATE      SERVICE
22/tcp    open      ssh
80/tcp    filtered  http
443/tcp   filtered  https

Nmap done: 1 IP address (1 host up) scanned in 1.27 seconds
root@srv1089520:~#
```

Figura 2.5. La única verificación válida del estado de los puertos se realiza desde fuera del servidor.

La bandera `-Pn` merece una nota: le indica a `nmap` que no intente primero averiguar si el anfitrión está vivo mediante un *ping*, sino que escanee directamente. Muchos proveedores descartan los *pings* entrantes, y sin esa bandera `nmap` concluiría que el servidor no existe y ni siquiera escanearía.

2.8.3. Cómo se evita el problema

Existen tres estrategias, en orden de preferencia.

Estrategia	Cómo	Cuándo se usa
No publicar el puerto	No declarar mapeo de puertos en absoluto	Servicios internos: bases de datos, cachés
Publicar solo en local	<code>-p 127.0.0.1:5432:5432</code>	Acceso puntual mediante túnel SSH
Reglas en <code>DOCKER-USER</code>	Escribir reglas de iptables en esa cadena	Casos excepcionales

La primera es la que se aplica en este módulo. Un servicio que no publica puertos sigue siendo perfectamente accesible para los demás servicios del mismo proyecto, a través de la red interna de Docker. Ese mecanismo es el contenido central de la Clase 5.

La segunda estrategia es la aplicación práctica de lo visto en la sección 2.6.1: al especificar 127.0.0.1 en el lado del anfitrión, la regla de traducción de direcciones solo acepta paquetes que ya venían de la propia máquina, y el problema del recorrido desaparece por construcción. Es la forma correcta de conectarse a la base con una herramienta de escritorio: se abre un túnel SSH, y para el servidor la conexión es local.

💡 PARA ENTENDER

Y acá está la conexión que quiero que quede: **la red interna no es una optimización, es la solución a este problema.** El Postgres de la Clase 5 no va a publicar el 5432 a ningún lado. La API le va a hablar por adentro. Y como el puerto nunca se publicó, no hay nada que un firewall tenga que proteger. El puerto más seguro es el que no existe.

2.9. Riesgos concretos de la exposición innecesaria

2.9.1. El descubrimiento es automático e inmediato

No hace falta que nadie conozca la dirección del servidor. El espacio completo de direcciones IPv4 son unos 4.300 millones de direcciones, y escanearlo entero por un puerto determinado lleva minutos con herramientas de uso corriente. Existen además buscadores especializados —Shodan, Censys— que mantienen un índice permanente de qué servicio corre en cada dirección del mundo.

Conviene detenerse en la aritmética, porque es la que vuelve intuitivo el resultado. Un solo equipo con conexión doméstica puede emitir del orden de un millón de paquetes por segundo; cubrir 4.300 millones de direcciones con un paquete cada una es cuestión de una hora larga, y repartido entre varias máquinas, de minutos. Escanear internet entera **no es una hazaña: es una tarea programada.** Y como el resultado se indexa y se vende, el atacante ni siquiera necesita escanear: le alcanza con consultar.

Un VPS recién creado empieza a recibir intentos de acceso dentro de la primera hora de existir.

🔧 EXPERIMENTO

Al final de la clase, con el servidor levantado hace un rato, corré:

```
sudo grep -c "Failed password" /var/log/auth.log
```

Van a ver decenas o cientos de intentos de login fallidos contra un servidor que existe hace tres horas y cuya dirección no le dieron a nadie. Es la demostración más contundente que tenés: **nadie te está atacando a vos. Están atacando a todos, todo el tiempo, automáticamente.** Y si el acceso fuera por contraseña, alguno entraría.

2.9.2. Qué se arriesga con cada puerto

Puerto	Servicio	Qué ocurre si se expone sin necesidad
22	SSH	Fuerza bruta continua. Con contraseña débil, compromiso total
3306	MySQL	Credenciales por defecto muy conocidas; lectura y borrado de datos
5432	PostgreSQL	Ídem

6379	Redis	Sin contraseña por defecto. Escritura arbitraria y ejecución remota
27017	MongoDB	Históricamente sin autenticación por defecto; miles de bases secuestradas
3000	Panel de administración	Contraseña en texto plano si no hay HTTPS
9000	Portainer y similares	Control total del motor de contenedores

Las filas de Redis y MongoDB ilustran un patrón de diseño que dominó una década y que hoy se considera un error: **el valor predeterminado permisivo**. Ambos productos nacieron pensados para correr en una red interna de confianza, y por eso arrancaban sin autenticación. Cuando la contenerización volvió trivial exponerlos a internet, ese predeterminado se convirtió en la causa de miles de bases secuestradas. Las versiones actuales lo corrigieron, pero la lección permanece y se enuncia en la sección 2.12.1.

2.9.3. Consecuencias que exceden al servidor

Conviene explicitar que el daño de un servidor comprometido rara vez se limita a los datos que contiene:

- **Minado de criptomonedas.** Consume el 100 % de los recursos y genera la factura.
- **Envío de correo no deseado.** La dirección IP queda en listas negras.
- **Participación en ataques de denegación de servicio contra terceros.** El servidor pasa a ser el atacante, y la responsabilidad es del titular.
- **Movimiento lateral.** Si desde ese servidor hay claves SSH hacia otros equipos, el compromiso se propaga.

El último punto es el que más frecuentemente se subestima, y es la razón concreta por la que la clave privada nunca debe copiarse al servidor. Un servidor comprometido con claves privadas adentro no es un servidor comprometido: es **todo lo que esas claves abren**, comprometido.

PARA ENTENDER

El razonamiento de "¿quién va a querer hackear mi calculadora?" está mal planteado. **A nadie le interesa tu calculadora.** Les interesa tu CPU, tu ancho de banda y tu dirección IP limpia. Tu aplicación es irrelevante; el servidor es el botón.

2.10. Publicación segura del panel de administración

Tras la instalación, el panel de Easypanel queda accesible en `http://IP_DEL_VPS:3000`. Esa configuración presenta dos problemas simultáneos:

1. **No hay cifrado.** La contraseña del administrador viaja en texto plano.
2. **El puerto está publicado por Docker**, con lo cual `ufw` no lo protege (sección 2.8).

La solución consiste en asignarle al panel un subdominio propio, aprovechando el registro comodín de la Clase 1:

1. Ingresar a **Settings → Domain** en Easypanel.
2. Configurar `easypanel.tudominio.com`.

3. Guardar y aguardar la emisión del certificado.
4. Verificar el acceso por `https://easypanel.tudominio.com`.
5. Recién entonces, bloquear el acceso directo por el puerto 3000.

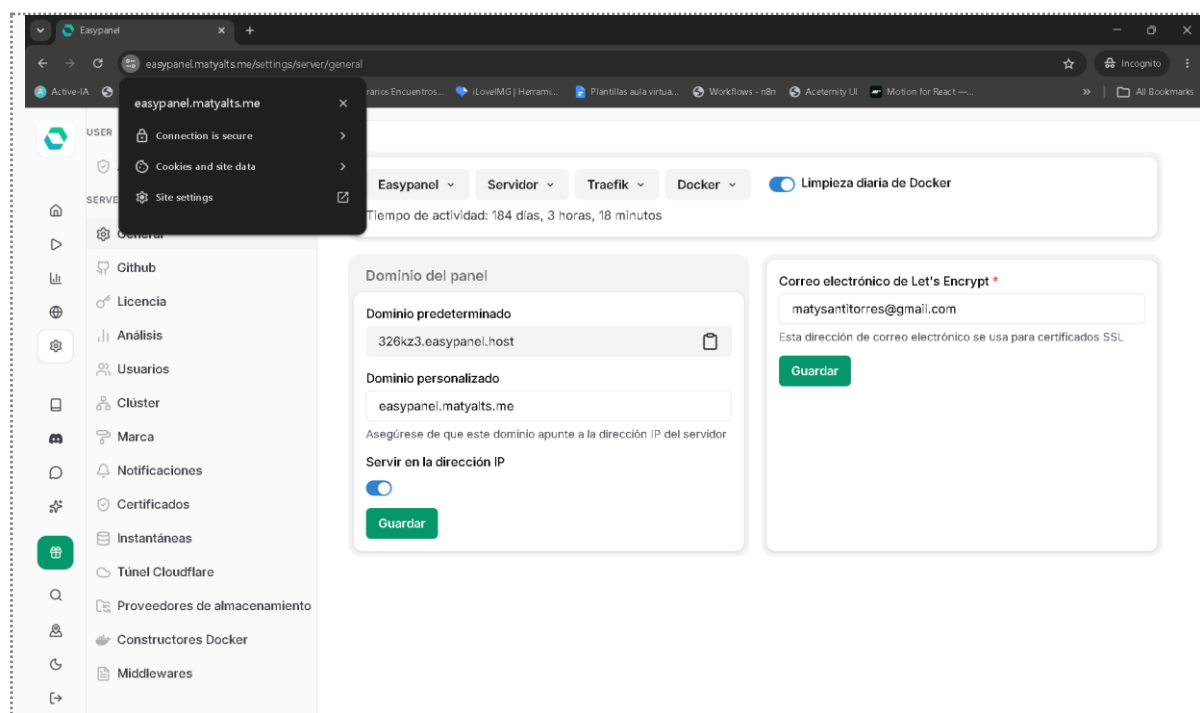


Figura 2.6. El panel de administración publicado bajo el dominio propio, con certificado válido.

Nótese que la operación no "mueve" el panel: el proceso sigue escuchando donde estaba. Lo que cambia es que ahora hay un proxy inverso adelante, que termina el cifrado y enruta por nombre —el mecanismo de alojamiento virtual de la sección 1.3—, y que el puerto directo deja de ser necesario. Es el mismo patrón que se aplicará a la aplicación entera en la Clase 4.

⚠ OJO ACÁ

El orden es sagrado: **primero verificás que el dominio nuevo entra, después cerrás el 3000**. Si lo hacés al revés y algo sale mal con el certificado, te quedaste sin panel y sin forma de arreglarlo desde el panel.

📌 DATO

Este paso es también la primera demostración práctica de para qué servía el registro comodín. Nadie cargó un registro easypanel en el panel DNS, y sin embargo easypanel.tudominio.com resuelve. Es exactamente el comportamiento descrito en la sección 1.12.1.

2.11. Endurecimiento mínimo

Las medidas siguientes constituyen la línea de base para cualquier servidor expuesto a internet. Ninguna es opcional en un entorno real.

#	Medida	Comando o ubicación
---	--------	---------------------

1	Autenticación solo por clave	PasswordAuthentication no en sshd_config
2	Firewall restrictivo	ufw default deny incoming
3	Verificación externa de puertos	nmap -Pn tudominio.com
4	Panel bajo HTTPS	Settings → Domain en Easypanel
5	Sistema actualizado	sudo apt update && sudo apt upgrade -y
6	Bloqueo de intentos repetidos	sudo apt install fail2ban
7	Actualizaciones automáticas de seguridad	sudo dpkg-reconfigure unattended-upgrades

La medida 7 merece un comentario, porque es la única de la lista que sigue trabajando cuando nadie mira. La enorme mayoría de los servidores comprometidos no cae por un ataque sofisticado sino por una vulnerabilidad **conocida, publicada y ya parcheada** que el administrador no aplicó. El intervalo entre que se publica una vulnerabilidad y que empieza a explotarse masivamente se mide en horas. Un servidor de práctica que nadie va a mirar durante la semana necesita esa automatización más, no menos, que uno atendido a diario.

2.11.1. fail2ban

`fail2ban` analiza los registros de acceso y bloquea temporalmente las direcciones que acumulan intentos fallidos. Con autenticación por clave el riesgo ya es bajo, pero la herramienta reduce además el ruido en los registros y el consumo de recursos.

```
sudo apt install fail2ban -y
sudo systemctl enable --now fail2ban
```

Estado de los bloqueos aplicados:

```
sudo fail2ban-client status sshd
```

Vale ubicar con precisión qué aporta y qué no. `fail2ban` **no impide un ataque dirigido**: quien tenga paciencia puede espaciar los intentos por debajo del umbral, o rotar direcciones de origen. Lo que hace es elevar el costo del ataque automatizado indiscriminado, que es la inmensa mayoría del tráfico hostil, y de paso mantener legibles los registros. Es una capa más, no una defensa: exactamente el concepto de la sección 2.12.1.

EXPERIMENTO

Volvé a correr `fail2ban-client status sshd` una semana después, al empezar la Clase 3. La cantidad de direcciones bloqueadas que se acumularon en siete días contra un servidor cuya dirección no le diste a nadie es difícil de creer hasta que la ves.

2.12. Principios de seguridad y evolución de las herramientas

Todo lo anterior fueron medidas concretas. Esta sección extrae los principios que hay detrás —que son pocos, viejos y aplicables mucho más allá de este práctico— y repasa hacia dónde evolucionan las herramientas utilizadas.

2.12.1. Tres principios que explican todo el capítulo

En 1975, Jerome Saltzer y Michael Schroeder publicaron un artículo que enunciaba ocho principios de diseño para sistemas seguros. Medio siglo después siguen vigentes, y tres de ellos son literalmente el resumen de este capítulo.

Mínimo privilegio. Todo componente debe operar con los permisos mínimos necesarios para su función, y ni uno más. Aplicaciones directas vistas acá: el contenedor de la base de datos no publica puertos porque no necesita ser alcanzable desde afuera; la clave pública instalada en el servidor no sirve para nada si el servidor cae, porque solo permite *verificar*, no *firmar*.

Valores predeterminados seguros. Lo que no está explícitamente permitido, se niega. Es literalmente la primera línea de la configuración del firewall (`default deny incoming`), y es lo que Redis y MongoDB hacían al revés en la sección 2.9.2, con los resultados conocidos.

Economía del mecanismo. Cuanto más simple es la protección, más fácil es verificar que funciona. Por eso "no publicar el puerto" es preferible a "publicarlo y filtrarlo con reglas": no hay que auditar ninguna regla, no hay que confiar en ninguna herramienta, no hay nada que se pueda configurar mal. El puerto no existe.

A esos tres se suma un concepto operativo posterior, la **defensa en profundidad**: ninguna capa se asume infalible, y por eso se apilan varias independientes. En este servidor hay al menos cuatro —clave en lugar de contraseña, firewall restrictivo, no publicar puertos internos, `fail2ban`— y cada una supone que las otras pueden fallar.

DATO

El artículo de Saltzer y Schroeder se llama *The Protection of Information in Computer Systems* y es de 1975. Está en línea, es gratuito y las primeras diez páginas se leen sin saber nada de criptografía. Que un texto de esa antigüedad describa con precisión los errores que se siguen cometiendo hoy dice algo sobre la disciplina: **las herramientas cambian todo el tiempo, los principios prácticamente nunca.**

2.12.2. De iptables a nftables

Las reglas que este capítulo describe se escriben, históricamente, con `iptables`. Desde 2014 existe su reemplazo, `nftables`, que unifica en un solo marco lo que antes eran cuatro herramientas separadas (`iptables`, `ip6tables`, `arptables`, `ebtables`), usa una sintaxis más regular y evalúa las reglas de forma más eficiente. Las distribuciones modernas —incluida Ubuntu 24.04— **ya usan nftables por debajo**: el comando `iptables` sigue existiendo pero es una capa de compatibilidad que traduce a `nftables`.

Para este práctico el cambio es invisible, porque `ufw` se ocupa de todo. Importa saberlo por dos motivos: al buscar documentación aparecen las dos sintaxis y conviene no mezclarlas, y al inspeccionar el estado real del sistema el comando actual es `sudo nft list ruleset`, que muestra —entre muchas otras cosas— las cadenas que Docker registró y de las que habla la sección 2.8.1.

2.12.3. Por qué Ed25519 y no RSA

La orden `ssh-keygen -t ed25519` de la sección 2.5.2 elige un algoritmo de firma concreto, y la elección no es arbitraria. **Ed25519** es un esquema de firma sobre curvas elípticas diseñado por Daniel Bernstein y colaboradores, normado para SSH en la RFC 8709. Frente al RSA de 4096 bits que todavía se ve en muchos tutoriales ofrece tres ventajas: claves mucho más cortas con seguridad equivalente o superior, firmas notablemente más rápidas, y —la más importante en la práctica— **muchas menos formas de configurarlo mal**. RSA admite tamaños de clave inseguros que siguen siendo aceptados por compatibilidad; Ed25519 tiene un solo tamaño y no hay perilla que tocar. Es economía del mecanismo aplicada a la criptografía.

2.13. Verificación

Las siete comprobaciones siguientes cierran el capítulo. Como en el capítulo anterior, no son variantes de la misma prueba: las cuatro primeras validan el acceso, las tres últimas validan el cierre, y solo una de todas ellas constituye evidencia externa.

#	Comprobación	Cómo	Resultado esperado
1	Acceso por clave	<code>ssh root@tudominio.com</code>	Entra sin pedir contraseña
2	Contraseña deshabilitada	Intentar desde un equipo sin clave	Acceso denegado
3	Los cuatro integrantes acceden	Cada uno desde su equipo	Todos entran
4	Firewall activo	<code>sudo ufw status verbose</code>	active, con 22, 80 y 443
5	Puertos reales desde afuera	<code>nmap -Pn tudominio.com</code>	Solo 22, 80 y 443
6	Panel por HTTPS	<code>https://easypanel.tudominio.com</code>	Candado válido
7	Puerto 3000 inaccesible	<code>http://IP_DEL_VPS:3000</code>	No responde

La comprobación 5 es la única que vale como prueba de que el servidor está cerrado. Las demás describen intenciones; esa describe hechos. Es la misma distinción que en la Clase 1 separaba "el panel muestra los registros que cargaste" de "los que el mundo consulta".

2.14. Errores frecuentes

Síntoma	Causa	Resolución
Se pierde el acceso tras habilitar ufw	Se habilitó antes de permitir el 22	Consola de emergencia de Hostinger
Se pierde el acceso tras deshabilitar contraseñas	La clave pública no estaba bien instalada	Consola de emergencia; revisar <code>authorized_keys</code>
ufw dice cerrado y nmap dice abierto	Puerto publicado por Docker (sección 2.8.1)	Dejar de publicarlo; usar red interna
El certificado del panel no se emite	El registro comodín no resuelve, o el 80 está cerrado	Revisar DNS y ufw
Un integrante entra y otro no	Falta su clave pública en el servidor	Agregarla a <code>authorized_keys</code>
Permission denied (publickey)	Se usó la clave privada de otro equipo	Cada uno usa su propio par
Advertencia grande de que cambió la clave del anfitrión	El servidor se reinstaló (o hay suplantación)	Verificar el motivo antes de borrar la línea de <code>known_hosts</code>
nmap no devuelve nada	El proveedor descarta los <i>pings</i>	Agregar la bandera <code>-Pn</code> (sección 2.8.2)

2.15. Actividades

Actividad 1 — Inventario de puertos. Ejecutar `ss -tlnp` y confeccionar una tabla con cada puerto en escucha, el proceso responsable, la interfaz en la que escucha y una justificación de por qué debe o no estar accesible desde internet. Indicar además a qué rango de la RFC 6335 pertenece cada uno (sección 2.6.2) y qué implica eso sobre los privilegios necesarios para abrirlo.

Actividad 2 — Contraste entre lo declarado y lo real. Comparar la salida de `ufw status verbose` con la de `nmap -Pn` ejecutado desde otro equipo. Documentar toda discrepancia y explicar su causa en términos del recorrido del paquete de la sección 2.7.1: por qué punto de enganche pasa el tráfico en cada caso.

Actividad 3 — Medición de la superficie de ataque. Contar los intentos de acceso fallidos acumulados desde la creación del servidor y estimar la tasa por hora. Identificar en el registro los diez países o rangos de origen más frecuentes y discutir qué dice esa distribución sobre la naturaleza del tráfico.

Actividad 4 — Revocación de acceso. Eliminar la clave pública de un integrante del archivo `authorized_keys`, verificar que pierde el acceso y que los demás lo conservan. Restaurarla después. Comparar el procedimiento con lo que habría requerido revocar el acceso de una sola persona bajo un esquema de contraseña compartida.

Actividad 5 — Investigación. Buscar en Shodan o Censys cuántas instancias de PostgreSQL están expuestas públicamente en Argentina. Discutir el resultado a la luz de la sección 2.9.2: ¿cuántas de esas exposiciones serían explicables por el problema de la sección 2.8?

Actividad 6 — Lectura del conjunto de reglas real. (*requiere acceso de administrador*) Ejecutar `sudo nft list ruleset` y localizar en la salida: las cadenas que creó `ufw`, las cadenas que creó Docker, y en qué punto de enganche está registrada cada una. Explicar, a partir de lo observado, por qué una regla de `ufw` no puede bloquear un puerto publicado por Docker.

Actividad 7 — El túnel SSH. (*opcional*) Investigar la opción `-L` de `ssh` y explicar cómo permitiría conectarse con una herramienta de escritorio a un servicio que escucha solo en `127.0.0.1` del servidor. Relacionar la respuesta con la segunda estrategia de la tabla de la sección 2.8.3 y con la distinción de interfaces de la sección 2.6.1.

2.16. Síntesis

1. Un VPS es una **ilusión sostenida por un hipervisor**: el aislamiento lo hace cumplir el procesador, los recursos se comparten, y el estado completo de la máquina es un archivo. De ahí salen el precio, la velocidad de reinstalación y el vecino ruidoso.
2. Con un VPS, la **administración del sistema y su seguridad son responsabilidad del titular**, no del proveedor. El corte no es comercial: el proveedor no puede ver adentro.
3. SSH autentica primero **al servidor** y después al cliente. La autenticación por clave es un desafío criptográfico: la clave privada nunca se transmite, y por eso un servidor comprometido no compromete el acceso.
4. Un socket se identifica por una **tupla**, no por un puerto. Un puerto abierto es **un proceso concreto escuchando en una interfaz concreta**, no una propiedad del servidor.
5. El firewall no es `ufw`: es **netfilter**, y evalúa las reglas en puntos de enganche distintos según el paquete sea para esta máquina o para otra.

6. **Docker se saltea ufw** porque su tráfico va por **FORWARD** y **ufw** escribe en **INPUT**. No es un error: es la consecuencia previsible del recorrido del paquete. La única verificación válida es escanear desde afuera.
7. El descubrimiento de servidores expuestos es **automático, permanente y no selectivo**. Nadie te ataca a vos; atacan a todos, todo el tiempo.
8. Mínimo privilegio, predeterminados seguros y economía del mecanismo son de 1975 y explican cada decisión de este capítulo. El puerto más seguro es **el que nunca se publicó**. De ahí la red interna de la Clase 5.

2.17. Referencias y lecturas complementarias

Sobre virtualización, el texto fundacional es G. Popek y R. Goldberg, *Formal Requirements for Virtualizable Third Generation Architectures* (*Communications of the ACM*, 17(7), 1974), donde se enuncian las tres propiedades —equivalencia, control de recursos y eficiencia— y se demuestra la condición que debe cumplir un juego de instrucciones. Para el caso de x86 y las soluciones que precedieron al soporte por hardware, el artículo de referencia es P. Barham et al., *Xen and the Art of Virtualization* (SOSP, 2003).

El protocolo SSH está normado en un conjunto de RFC del IETF, todas de acceso libre en [rfc-editor.org](https://www.rfc-editor.org): la **RFC 4251** describe la arquitectura general, la **RFC 4252** la capa de autenticación, la **RFC 4253** la capa de transporte y la **RFC 4254** la capa de conexión; la **RFC 8709** incorpora Ed25519 como algoritmo de firma. La asignación de puertos y sus tres rangos está normada en la **RFC 6335**; el protocolo TCP, cuya especificación original era la RFC 793 de 1981, fue consolidado en 2022 en la **RFC 9293**.

En seguridad, la lectura obligada es J. Saltzer y M. Schroeder, *The Protection of Information in Computer Systems* (*Proceedings of the IEEE*, 63(9), 1975): ocho principios de diseño enunciados hace medio siglo que siguen describiendo con exactitud los errores actuales. Para un tratamiento moderno y extenso, R. Anderson, *Security Engineering* (3.ª edición, Wiley, 2020), disponible gratuitamente en el sitio del autor, es la referencia estándar.

Como manual operativo de administración de sistemas Unix, E. Nemeth et al., *UNIX and Linux System Administration Handbook* (5.ª edición, Addison-Wesley, 2017) cubre con profundidad SSH, netfilter y el endurecimiento de servidores. Para la capa de transporte y el modelo de sockets, el capítulo correspondiente de Kurose y Ross, *Computer Networking: A Top-Down Approach* (8.ª edición, Pearson, 2021), continúa el enfoque descendente del capítulo anterior.